



Objetos em Python



Eduardo Ogasawara

<https://eic.cefet-rj.br/~eogasawara>



O que é uma Variável

Uma variável é um **nome associado a um valor**. Esse valor fica armazenado na memória do computador. O nome permite acessar, reutilizar e modificar o dado.

Quando atribuímos um valor a uma variável, estamos criando uma referência na memória que podemos usar sempre que precisarmos daquele dado.



```
idade = 30
```

```
idade
```

```
# 30
```

```
idade = idade + 1
```

```
idade
```

```
# 31
```

Objetos, Tipos e Valores

Valor

O conteúdo armazenado pelo objeto

Tipo

Define quais operações podem ser feitas

Operações

Funções que produzem novos objetos

Todo objeto em Python possui um **valor** e um **tipo**. O tipo determina o comportamento do objeto e quais operações são permitidas sobre ele.

```
a = 5
b = 2
c = a * b
type(c)
# <class 'int'>
```

Data Frames (Tabelas)

Data Frames são estruturas tabulares fundamentais para análise de dados. Eles combinam várias colunas em uma única estrutura, onde cada coluna pode ter um tipo de dado diferente. São amplamente utilizados em ciência de dados para organizar, manipular e analisar informações de forma eficiente.

Estrutura organizada

Linhas representam registros individuais, colunas representam atributos ou características

Tipos flexíveis

Cada coluna pode conter strings, números, datas ou outros tipos de dados

Análise poderosa

Permite filtrar, agrupar e calcular estatísticas

```
import pandas as pd
df = pd.DataFrame({
    "face": ["ás", "dois", "quatro"],
    "naipe": ["ouros", "copas", "paus"],
    "valor": [1, 2, 4]
})
print(df)
```

Salvando e Lendo Arquivos

Data Frames podem ser facilmente exportados e importados em arquivos CSV, permitindo compartilhar e persistir dados entre análises.

Salvar em CSV

```
df.to_csv("cartas.csv",  
         index=False)
```

Ler do CSV

```
cartas = pd.read_csv(  
    "cartas.csv")
```

Compartilhar

Dados persistidos podem ser compartilhados entre projetos e colaboradores

Valores Ausentes e Especiais

Python oferece diferentes maneiras de representar a ausência de informação, cada uma com comportamento específico em cálculos e operações.

None

Indica a ausência completa de um objeto. Exemplo: `x = [1, None, 3]`

np.nan

Representa um número ausente. Exemplo: `y = np.array([1.0, np.nan, 3.0])`

❏ **Importante:** `None` e `np.nan` não são intercambiáveis. Use `None` para objetos Python genéricos e `np.nan` para operações numéricas com NumPy.

Lidando com Valores Ausentes

Valores ausentes podem contaminar cálculos estatísticos, mas Python oferece funções especializadas que ignoram valores inválidos.

⚠ Problema

```
y = [1.0, np.nan, 3.0]
np.mean(y)
# Resultado: nan
```

A função padrão retorna `nan` quando encontra valores ausentes, invalidando todo o cálculo.

✓ Solução

```
np.nanmean(y)
# Resultado: 2.0
```

Funções com prefixo `nan` ignoram valores ausentes e calculam corretamente a média dos valores válidos.

Identificar valores ausentes

```
np.isnan(y)
# array([False,  True, False])
```

Usar funções nan-aware

Use `nanmean`, `nansum`, `nanstd` para cálculos robustos

Remover ou preencher valores

Em tabelas pandas, use `dropna()` para remover ou `fillna()` para preencher valores ausentes.

O que são Objetos

Dados Simples

Números inteiros, decimais e textos que representam valores únicos

```
numero = 10  
type(numero)  
# <class 'int'>
```

Estruturas

Listas, dicionários e outras coleções que agrupam múltiplos valores

```
lista = [1, 2, 3]  
type(lista)  
# <class 'list'>
```

Comportamento

Cada tipo tem operações específicas e características próprias

```
texto = "dados"  
texto.upper()  
# 'DADOS'
```

Em Python, variáveis armazenam objetos de vários tipos. Cada tipo tem um comportamento específico que define como podemos trabalhar com ele.

Vetores (Sequências Numéricas)

Arrays do NumPy armazenam múltiplos valores do mesmo tipo de forma eficiente, sendo a base da programação científica em Python.

```
import numpy as np
```

```
dado = np.array([1, 2, 3, 4, 5, 6])
```

```
dado
```

```
# array([1, 2, 3, 4, 5, 6])
```

Escalar vs Array de Tamanho 1

Escalar

Um número solto (5) é um escalar. Ele não tem tamanho: `len(5)` gera `TypeError`.

Array de tamanho 1

`np.array([5])` é um vetor com um único elemento: `len()` devolve 1.

Verificação

Use `len()` ou `.size` para arrays e `.ndim` para ver o número de dimensões.

```
import numpy as np

numero = 5
# len(numero) # TypeError: object of type 'int' has no len()

vetor = np.array([5])
len(vetor) # 1

dado = np.array([1, 2, 3, 4, 5, 6])
len(dado) # 6
```

Tipo Inteiro e Texto

Inteiros (int)

Números inteiros para contagens e índices

```
inteiro = 1  
type(inteiro)  
# <class 'int'>
```

Textos (str)

Strings definidas entre aspas

```
texto = "ás"  
type(texto)  
# <class 'str'>
```

Python distingue claramente entre números inteiros e textos. Cada tipo tem operações específicas e comportamentos diferentes.

Sequências e Funções

Sequências podem armazenar números ou textos. Funções especializadas operam sobre essas sequências, permitindo análises e transformações dos dados.

Sequências Numéricas

```
import numpy as np
cartas = np.arange(1, 14)
np.sum(cartas)
# 91
```

Funções matemáticas retornam valores agregados

Sequências de Texto

```
faces = ["ás", "dois", "três", "quatro", "cinco", "seis", "sete",
"oito", "nove", "dez", "valete", "dama", "rei"]
max(faces)
# 'ás'
sorted(faces)[:3]
# ['cinco', 'dama', 'dez']
```

Textos são comparados pela ordem dos caracteres (Unicode): letras acentuadas, como 'á', vêm depois de 'z'. Por isso `max(faces)` é 'ás'.

Tipo Decimal (float)

Em Python, números decimais usam **ponto flutuante** de dupla precisão (float64, cerca de 15 a 16 dígitos significativos). Mesmo inteiros podem ser armazenados como float dependendo do contexto.

```
dados = np.array([1, 2, 3, 4, 5, 6],  
dtype=float)  
dados  
# array([1., 2., 3., 4., 5., 6.])  
  
dados.dtype  
# dtype('float64')
```

⚠ Atenção: float tem erros de arredondamento (0.1 + 0.2 == 0.3 é False). Para valores monetários, use decimal.Decimal.

Tipo Lógico (Booleano)

Comparações

Operações de comparação retornam True ou False

Decisões

Valores booleanos controlam fluxo de execução

Filtros

Permitem selecionar subconjuntos de dados

Valores lógicos representam **verdadeiro ou falso**. Eles são resultado de comparações e essenciais para decisões e filtros em programação.

```
logico = np.array([True, False, 3 >= 4, 3 < 4, 3 != 4, 4 == 4])  
logico  
# array([ True, False, False,  True,  True,  True])
```

Números Complexos

Python suporta **números complexos** nativamente. Eles possuem parte real e parte imaginária, representados com o sufixo `j`.

São amplamente usados em ciência, engenharia, processamento de sinais e física quântica.

```
comp = np.array([1+1j, 1+2j, 1+3j])
```

```
comp
```

```
# array([1.+1.j, 1.+2.j, 1.+3.j])
```

```
comp.dtype
```

```
# dtype('complex128')
```

Tipo Byte (Dados Brutos)

Representação

Bytes representam dados binários de baixo nível, essenciais para arquivos e comunicação

Intervalo

Cada byte pode assumir valores entre 0 e 255 — 8 bits

Tratamento

Valores fora do intervalo precisam ser convertidos explicitamente

```
r = bytearray(3)
r[1] = 255
# r[2] = 1024 # gera erro: valor fora do intervalo
r[2] = 1024 % 256 # conversão explícita para 0
r
# bytearray(b'\x00\xff\x00')
```


Rótulos e Atributos

Em arrays, rótulos podem ser simulados com dicionários, facilitando a interpretação dos dados.

Dados Rotulados (pandas.Series)

```
import numpy as np
import pandas as pd
dado = np.array([1, 2, 3, 4, 5, 6])
nomes = ["um", "dois", "três", "quatro", "cinco", "seis"]
rotulado = pd.Series(dado, index=nomes)
rotulado["três"] # 3
```

Atributos do Objeto

```
dado.dtype # dtype('int64')
dado.shape # (6,)
dado.ndim # 1
```

Rótulos dão significado aos dados. Atributos (acessados com ponto) descrevem o próprio objeto: tipo, forma e dimensões.

Transformando em Matriz

Preenchimento por Coluna

Com `order="F"`, `reshape()` preenche a matriz coluna por coluna. O padrão do NumPy (`order="C"`) preenche linha por linha.

```
import numpy as np
dado = np.arange(1, 7)
# preenchimento por coluna (order="F")
m = dado.reshape((2, 3), order="F")
m
# array([[1, 3, 5],
#        [2, 4, 6]])
```

Reshape Explícito

A função `reshape()` reorganiza os dados na forma desejada

Criando Matrizes

A ordem de preenchimento **muda o significado dos dados**.

Por Coluna (order="F")

```
dados = np.arange(1, 7)
por_coluna = dados.reshape((2, 3),
                           order="F")
por_coluna
# array([[1, 3, 5],
#        [2, 4, 6]])
```

Por Linha (order="C")

```
por_linha = dados.reshape((2, 3),
                          order="C")
por_linha
# array([[1, 2, 3],
#        [4, 5, 6]])
```

Tipo e Classe dos Objetos

Exemplo:

```
dado = np.arange(1, 7).reshape((2, 3))
```

Tipo Interno

Define a estrutura básica do objeto

```
type(dado)  
# <class 'numpy.ndarray'>
```

Forma

Influencia como os dados são usados e acessados

```
dado.shape  
# (2, 3)
```

Comportamento

A estrutura define as operações disponíveis

Um objeto possui um tipo interno e um comportamento específico. Em arrays, a forma determina como os dados são organizados e manipulados.

Data e Hora

Python possui tipos específicos para **datas e horas**, fundamentais em séries temporais e análises cronológicas. O módulo `datetime` oferece classes completas para manipular datas, horas e fusos horários.

```
from datetime import datetime

now = datetime.now()
now
# datetime.datetime(2024, 1, 15,
# 14, 30, 45, 123456)

type(now)
# <class 'datetime.datetime'>
```

Dados Categóricos

Representação

Dados categóricos representam **classes ou grupos** fixos, como gênero, região ou categoria de produto

Análise

Facilitam análises estatísticas e visualizações por grupo

Eficiência

Ocupam menos memória que texto simples ao reutilizar categorias

```
import pandas as pd

genero = pd.Categorical(["feminino", "masculino", "feminino", "masculino"])
genero
# ['feminino', 'masculino', 'feminino', 'masculino']
# Categories (2, object): ['feminino', 'masculino']
```

Conversões de Tipo

Objetos podem ser **convertidos entre tipos** diferentes, permitindo combinar dados e realizar operações específicas.

Booleano → Inteiro

```
int(True) # 1
int(False) # 0
```

Booleano → Decimal

```
float(False) # 0.0
float(True) # 1.0
```

Inteiro → Booleano

```
bool(1) # True
bool(0) # False
```

Qualquer → Texto

```
str(1) # '1'
str(True) # 'True'
```

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Introdução essencial à programação e pensamento computacional com Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media.

Guia completo das ferramentas essenciais para trabalhar com dados em Python.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media.

Aprenda ciência de dados construindo algoritmos do zero com Python.